Projet de remplacement de stage — L3 Informatique et Systèmes

Conception d'un Mini-SOC Low-Cost pour PME Togolaises

Moteur de corrélation stateful en Python et évaluation comparative avec Wazuh

Adrien
École Polytechnique de Lomé (EPL)
L3 Informatique et Systèmes

Mars 2026

Résumé

Ce projet propose la conception et l'implémentation d'un *mini-SOC open-source* adapté aux contraintes des PME togolaises : budget limité, ressources matérielles restreintes, et pénurie de compétences en cybersécurité. Le système comprend un pipeline de collecte et normalisation de logs, un moteur de corrélation stateful développé en Python avec règles externalisées en YAML, et deux scénarios d'attaque simulés représentatifs des menaces locales. La solution est rigoureusement évaluée et comparée à Wazuh, solution industrielle open-source de référence, selon des métriques de détection (taux vrais/faux positifs), de latence et de consommation de ressources. Les livrables incluent un dépôt Git reproductible (IaC), un rapport analytique et une démonstration technique en direct. Ce projet s'inscrit dans le contexte de la Stratégie Nationale de Cybersécurité du Togo 2024–2028 et vise à démontrer la faisabilité d'une approche de détection à coût quasi-nul pour les organisations de la sous-région.

Note méthodologique importante

Les chiffres de performance présentés dans ce rapport (taux de détection, latence, RAM) sont des **objectifs de conception**, pas des résultats mesurés. Les mesures réelles seront collectées lors des phases de test (semaines 5–8) et viendront remplacer ces valeurs cibles dans le rapport final.

Table des matières

1 Introduction **3**
1.1 Contexte togolais . . . 3
1.1.1 Données sourcées . . . 3
1.1.2 Constats observationnels . . . 3
1.2 Problématique . . . 4
1.3 Hypothèses de travail . . . 4

2 État de l'art **4**
2.1 Panorama des solutions existantes 4
2.2 Wazuh — comparant de référence 5
2.3 Lacune identifiée 5

3 Architecture et méthodologie **5**
3.1 Vue d'ensemble du pipeline 5
3.2 Laboratoire virtualisé (Infrastructure as Code) 5
3.2.1 Topologie réseau 5
3.2.2 Allocation mémoire 6
3.3 Scénarios d'attaque retenus 6
3.3.1 Scénario 1 — Brute-force SSH suivi d'une connexion réussie 6
3.3.2 Scénario 2 — Reconnaissance réseau suivie d'une exfiltration 7
3.4 Architecture du moteur Python 7
3.4.1 Environnement technique et dépendances 7
3.4.2 Choix du backend d'états 8
3.4.3 Structure des règles YAML 10
3.4.4 Architecture du moteur 10

4 Plan d'implémentation — 10 semaines **10**

5 Métriques d'évaluation **11**
5.1 Métriques de détection 11
5.2 Métriques de performance 12
5.3 Objectifs de conception (valeurs cibles) 12

6 Impact et perspectives locales **12**
6.1 Adéquation au contexte togolais 12
6.2 Limites 13
6.3 Perspectives 13

7 Conclusion **13**

A Structure du dépôt Git **15**

B Règles de corrélation — Scénario 2 **15**

C Bibliographie — Sources à compléter **16**

Introduction

Contexte togolais

Données sourcées

L'Union Internationale des Télécommunications (UIT) publie annuellement l'indice mondial de cybersécurité (GCI), qui positionne le Togo parmi les pays africains en progression sur les cinq piliers évalués (cadre légal, capacités techniques, organisation institutionnelle, renforcement des capacités, coopération internationale) [International Telecommunication Union, 2024]. L'Agence Nationale de la Cybersécurité (ANCy), créée en 2020, constitue le principal organe de coordination nationale en matière de sécurité numérique.

Sur le plan des coûts, les tarifs publics des solutions SIEM commerciales permettent d'établir un constat objectif et vérifiable : une licence Splunk Enterprise débute à plusieurs dizaines de milliers de dollars annuels, et les solutions EDR professionnelles (CrowdStrike Falcon, SentinelOne) facturent entre 50 et 150 USD par poste et par an. Ces coûts sont structurellement incompatibles avec les capacités budgétaires des petites et moyennes entreprises de la sous-région.

Constats observationnels

Les éléments suivants sont présentés comme des constats qualitatifs, non comme des statistiques mesurées, en l'absence de sources académiques publiées suffisamment granulaires sur le tissu des PME togolaises :

- **Coût des solutions industrielles.** Les SIEM commerciaux (Splunk, IBM QRadar) et les EDR professionnels dépassent structurellement les capacités budgétaires des PME de la sous-région, indépendamment de toute statistique précise sur leur taux d'adoption.
- **Pénurie de compétences locales.** Les formations spécialisées en cybersécurité opérationnelle (Blue Team, analyse forensique, ingénierie SOC) sont peu développées en Afrique de l'Ouest francophone ; l'EPL Lomé constitue l'une des rares institutions proposant une filière informatique structurée au Togo.
- **Menaces peu couvertes localement.** Les vecteurs d'attaque les plus fréquents dans le contexte ouest-africain — phishing via WhatsApp et SMS, fraude aux services Mobile Money (Flooz, T-Money), ransomwares ciblant des systèmes Windows non patchés — sont peu représentés dans les bases de règles des solutions génériques internationales, conçues pour des contextes d'entreprise occidentaux.

Positionnement méthodologique sur les données contextuelles

Ce rapport distingue explicitement deux niveaux de données : **(1) données sourcées**, étayées par des publications officielles citées en bibliographie ; **(2) constats observationnels**, présentés comme tels sans prétention statistique. Cette distinction est intentionnelle : introduire des chiffres précis sans source vérifiable dans un rapport académique constitue une faute méthodologique. Si des rapports ANCy ou des études CEDEAO accessibles sont identifiés avant la soutenance, ils remplaceront les constats observationnels correspondants.

Problématique

Question centrale

Est-il possible de construire un système de détection d'intrusion stateful, capable d'identifier des chaînes d'attaques complexes (kill-chains), avec des contraintes matérielles compatibles avec les PME togolaises ($\leq$8 Go RAM, coût logiciel nul), et dont les performances de détection sont comparables à celles de Wazuh sur les mêmes scénarios ?

Hypothèses de travail

1. **H1 — Équivalence de détection.** Le moteur Python atteint un taux de vrais positifs (TPR) $\geq 90\,\%$ et un taux de faux positifs (FPR) $\leq 10\,\%$ sur les scénarios définis, sans différence statistiquement significative avec Wazuh sur ces mêmes scénarios.
2. **H2 — Supériorité en ressources.** Le moteur Python consomme moins de 4 Go de RAM en charge et présente une latence de détection inférieure à celle de Wazuh sur le même matériel.
3. **H3 — Adaptabilité locale.** Les règles de corrélation peuvent être écrites et modifiées par un utilisateur avec des compétences Python intermédiaires, sans expertise SIEM spécialisée.

État de l'art

Panorama des solutions existantes

Table 1. Comparaison des catégories de solutions de détection

Solution	Coût	RAM min.	Pertinence contexte PME Togo
Splunk Enterprise	Très élevé	16 Go+	Hors de portée financière
IBM QRadar	Très élevé	24 Go+	Hors de portée financière
Wazuh	Gratuit	4 Go+	Viable techniquement, lourd en ressources
Suricata (IDS réseau)	Gratuit	1 Go+	Détection réseau uniquement, pas de corrélation
Antivirus basiques	Variable	Faible	Aucune corrélation, pas de détection réseau
Mini-SOC Python	**Gratuit**	**<4 Go**	**Cible de ce projet**

Pourquoi Norton est exclu du benchmark

Un antivirus grand public (Norton, McAfee, Avast) est fondamentalement différent d'un moteur de corrélation stateful : il opère sur des signatures de fichiers et des heuristiques comportementales sur un seul poste, sans capacité de corréler des événements multi-sources dans le temps. Comparer ces deux catégories d'outils n'aurait pas de sens

technique. Le benchmark retenu dans ce projet est **Wazuh uniquement**, qui appartient à la même catégorie fonctionnelle (SIEM/corrélation d'événements).

Wazuh — comparant de référence

Wazuh est un SIEM open-source dérivé d'OSSEC, activement maintenu, utilisé en production dans des milliers d'organisations. Il comprend un agent déployé sur les hôtes supervisés, un manager centralisant les événements, et un indexer (OpenSearch) pour le stockage et la recherche.

Ses atouts pour ce projet : gratuité totale, présence de règles de détection pré-écrites, documentation exhaustive, et usage croissant en Afrique dans des structures à budget limité. Sa limite principale : l'empreinte mémoire, notamment due à OpenSearch, qui nécessite au minimum 4 à 8 Go de RAM en configuration productive.

Lacune identifiée

Aucun outil de corrélation stateful n'existe à ce jour qui soit : (a) entièrement open-source et gratuit, (b) déployable sur moins de 4 Go de RAM, (c) dont les règles de détection soient lisibles et modifiables par un développeur Python sans formation SIEM, (d) documenté avec des scénarios de menaces propres au contexte ouest-africain.

Ce projet adresse directement ce manque.

Architecture et méthodologie

Vue d'ensemble du pipeline

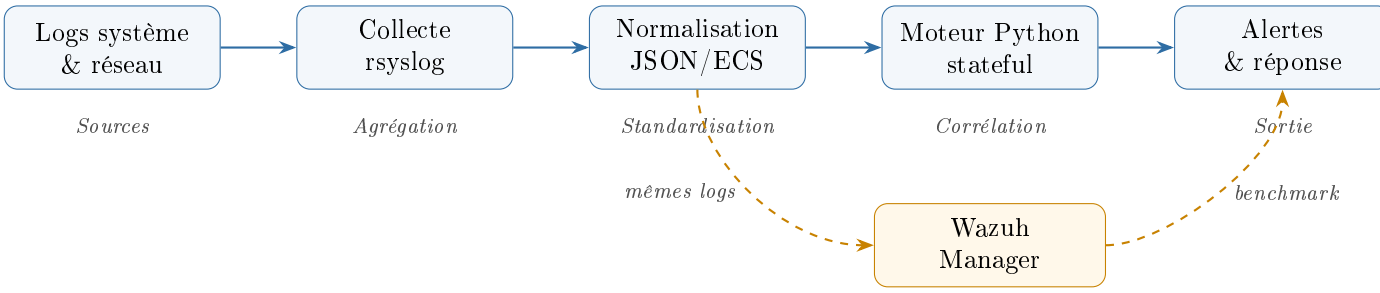

Figure 1. Pipeline Mini-SOC et position du benchmark Wazuh

Laboratoire virtualisé (Infrastructure as Code)

Topologie réseau

Le laboratoire est entièrement isolé de l'Internet (réseaux VirtualBox en mode `Host-Only` et `Internal Network`). La topologie retenue est la suivante :

Table 2. Composants du laboratoire

Composant	Type	IP	Rôle
Firewall / Routeur	VM Alpine	192.168.100.2	Routage, segmentation, filtrage
Serveur collecte + moteur	Conteneur	192.168.100.10	Pipeline logs, moteur Python
Cible SSH	Conteneur	192.168.100.20	Serveur SSH cible des attaques
Machine attaquante	VM Kali	192.168.100.50	Simulation des scénarios
Wazuh Manager/Indexer	VM Debian	192.168.100.30	Benchmark (phases 8–10)

Allocation mémoire

Avec 16 Go de RAM disponibles sur l'hôte Xubuntu, l'allocation est :

Table 3. Budget RAM du laboratoire

Composant	RAM allouée	Phase active
Firewall Alpine	512 Mo	1–10
Serveur collecte + moteur	1,5 Go	1–10
Cible SSH	512 Mo	1–10
Machine attaquante Kali	1 Go	1–10
Wazuh Manager	2 Go	8–10
Wazuh Indexer (OpenSearch)	2 Go	8–10
Total phases 1–7	**3,5 Go**	—
Total phases 8–10	**7,5 Go**	—
Marge système hôte	**≥8 Go**	—

Scénarios d'attaque retenus

Décision de scope — 2 scénarios uniquement

Le projet se limite à deux scénarios d'attaque définis dès la semaine 0 et figés jusqu'à la soutenance. Cette décision est délibérée : deux scénarios maîtrisés, bien documentés, avec des métriques précises, ont plus de valeur qu'une liste de scénarios bâclés. Tout scénario supplémentaire éventuel sera traité en annexe et clairement étiqueté « hors périmètre principal ».

Scénario 1 — Brute-force SSH suivi d'une connexion réussie

Vecteur local : Attaque automatisée sur des serveurs SSH exposés, fréquente dans le contexte des petites structures togolaises dont les services sont directement accessibles via IP publique sans VPN.

Séquence d'événements :

1. Multiples tentatives de connexion SSH échouées depuis la même IP source (≥ 10 tentatives en moins de 60 secondes).

2. Connexion SSH réussie depuis la même IP source dans les 120 secondes suivant la dernière tentative échouée.

Règle de corrélation formelle :

$$\exists\, t_0 : \left|\{e \mid e.type = \texttt{ssh_fail},\ e.src = s,\ t_0 \le e.t < t_0{+}60\}\right| \ge 10 \wedge \exists\, t_1 : e.type = \texttt{ssh_ok},\ e.src = s,\ t_0 \le t_1 \le$$

Scénario 2 — Reconnaissance réseau suivie d'une exfiltration

Vecteur local : Attaque en deux temps visant des serveurs de fichiers internes, représentative des intrusions observées dans les PME de la sous-région (accès initial via credential faible, puis exfiltration vers service cloud ou IP externe).

Séquence d'événements :

1. Scan réseau large (SYN scan ou UDP scan) depuis une IP interne vers au moins 20 hôtes en moins de 30 secondes.
2. Transfert de données sortant vers une IP externe (≥ 10 Mo) dans les 300 secondes suivant le scan.

Règle de corrélation formelle :

$$\exists\, t_0 : \left|\{e \mid e.type = \texttt{scan},\ e.src = s,\ t_0 \le e.t < t_0{+}30\}\right| \ge 20 \wedge \exists\, t_1 : e.type = \texttt{transfer_out},\ e.size \ge 10^7,\ e.sr$$

Architecture du moteur Python

Environnement technique et dépendances

Le moteur est développé et testé sous **Python 3.12** (version installée par défaut sur Ubuntu 24.04 LTS / Xubuntu 24.04, environnement de développement de ce projet). La version minimale requise est Python 3.10 (introduction des *union types* `X | Y` utilisés dans les annotations). Toutes les dépendances sont fixées dans `requirements.txt` à la racine du dépôt :

Listing 1. requirements.txt – dépendances exactes du moteur

```
# requirements.txt -- Python 3.12 (min. 3.10)
pyyaml>=6.0          # chargement et parsing des regles YAML
watchdog>=4.0        # surveillance inotify des fichiers de logs
jsonschema>=4.0      # validation du schema JSON des evenements
pytest>=8.0          # tests unitaires du moteur FSM
```

Table 4. Stack technique du moteur Python – rôle de chaque dépendance

Composant	Version	Rôle précis dans le moteur
Python	3.12 (min. 3.10)	Langage principal ; annotations de type `dict | None` requièrent 3.10+
PyYAML	≥ 6.0	Chargement des règles `*.yaml` depuis `engine/rules/` au démarrage
watchdog	≥ 4.0	Surveillance en temps réel du répertoire de logs via `inotify` (Linux) ; déclenche le parser à chaque nouvelle ligne
jsonschema	≥ 4.0	Validation du schéma des événements normalisés avant injection dans la FSM
pytest	≥ 8.0	Tests unitaires : rejoue des séquences d'événements et vérifie les alertes générées

Choix du backend d'états

Deux options ont été évaluées pour le stockage des états de corrélation.

Option A — Redis (écarté). Redis offre TTL natif, pub/sub et persistence RDB/AOF. Il est écarté pour trois raisons : (1) dépendance externe non justifiée pour un moteur mono-serveur gérant deux scénarios ; (2) overhead mémoire de 50–200 Mo inutile pour un lab ; (3) états opaques sans `redis-cli`, ce qui alourdit les tests unitaires. Cette décision est réversible : si le projet évolue vers plusieurs sources de logs simultanées, Redis sera réévalué.

Option B — Dictionnaire Python avec persistence JSON (retenu). Le moteur stocke les états actifs dans un dictionnaire Python en mémoire. **Le risque principal est la perte d'état au redémarrage** : si le processus plante, les attaques en cours de corrélation sont perdues.

La réponse retenue est une *persistence périodique sur JSON* avec reprise automatique au démarrage, implémentée dans la classe `StateStore` décrite ci-dessous. La fenêtre de perte maximale est bornée à 30 secondes (paramétrable), ce qui est acceptable au regard des fenêtres de détection de 60 à 300 secondes définies dans les règles YAML.

Listing 2. engine/state_store.py – Persistence JSON avec reprise au redémarrage

```python
# engine/state_store.py  --  Python 3.12
import json, time, threading, pathlib

SNAPSHOT_PATH = pathlib.Path("engine/state_snapshot.json")
SNAPSHOT_INTERVAL = 30  # secondes entre chaque sauvegarde (
    parametrable)

class StateStore:
    """Dictionnaire d'etats avec persistence JSON periodique.

    Au demarrage : recharge le snapshot existant et purge
    les etats dont la fenetre temporelle est depassee.
    En fonctionnement : sauvegarde toutes les SNAPSHOT_INTERVAL
        secondes
    dans un thread daemon (n'empeche pas l'arret propre du
        processus).
    Perte maximale en cas de crash : SNAPSHOT_INTERVAL secondes d'
        etats.
    """

    def __init__(self):
        self._states: dict = {}
        self._lock = threading.Lock()
        self._load()           # reprise au demarrage
        self._start_loop()     # sauvegarde periodique en arriere-
            plan

    # -- Persistence
        ---------------------------------------------------

    def _load(self) -> None:
        """Recharge le snapshot et purge les etats expires."""
        if not SNAPSHOT_PATH.exists():
            return
        with open(SNAPSHOT_PATH) as f:
```

```
            raw: dict = json.load(f)
        now = time.time()
        # Ne conserve que les etats dont la fenetre n'est pas
            depassee
        self._states = {
            k: v for k, v in raw.items()
            if now - v["last_seen"] < v["window_seconds"]
        }

    def _save(self) -> None:
        """Ecrit le snapshot sur disque (ecriture atomique via tmp)
            ."""
        tmp = SNAPSHOT_PATH.with_suffix(".tmp")
        with self._lock:
            with open(tmp, "w") as f:
                json.dump(self._states, f, indent=2)
        tmp.replace(SNAPSHOT_PATH)  # atomique sur POSIX

    def _start_loop(self) -> None:
        """Lance le thread de sauvegarde periodique (daemon)."""
        def loop():
            while True:
                time.sleep(SNAPSHOT_INTERVAL)
                self._save()
        threading.Thread(target=loop, daemon=True).start()

    # -- Interface publique
        ------------------------------------------

    def get(self, key: str) -> dict | None:
        return self._states.get(key)

    def set(self, key: str, value: dict) -> None:
        with self._lock:
            self._states[key] = value

    def delete(self, key: str) -> None:
        with self._lock:
            self._states.pop(key, None)
```

Ce que ce mécanisme garantit et ne garantit pas

Garantit :

- Reprise des états non expirés au redémarrage (crash ou mise à jour).
- Fenêtre de perte bornée à 30 s (paramétrable via `SNAPSHOT_INTERVAL`).
- Purge automatique des états dont la fenêtre temporelle est dépassée.
- Écriture atomique via fichier temporaire + `os.replace` : pas de snapshot corrompu en cas de crash pendant la sauvegarde.

Ne garantit pas :

- Scalabilité multi-processus (limitation connue, hors périmètre lab).
- Durabilité sub-seconde (Redis serait requis).

Conclusion : approche acceptable pour le périmètre mono-serveur de ce projet. Décision documentée dans `docs/decisions.md` du dépôt Git.

Structure des règles YAML

Les règles de corrélation sont externalisées dans des fichiers YAML, indépendants du code du moteur. Un exemple de règle pour le scénario 1 :

Listing 3. Règle YAML — Scénario 1 : brute-force SSH

```yaml
# rules/ssh_bruteforce.yaml
id: SSH_BRUTEFORCE_001
name: "Brute-force SSH suivi d'une connexion reussie"
description: >
  Detecte une succession de tentatives SSH echouees
  suivie d'une connexion reussie depuis la meme source.
# -- Etape 1 : accumulation d'echecs --
trigger:
  event_type: ssh_auth_failed
  threshold: 10           # >= 10 echecs
  window_seconds: 60      # dans une fenetre de 60s
  group_by: src_ip        # par IP source
# -- Etape 2 : confirmation --
escalation:
  event_type: ssh_auth_success
  same_field: src_ip      # meme IP source
  within_seconds: 180     # dans les 3 minutes suivantes
# -- Sortie --
alert:
  severity: HIGH
  mitre_tactic: "TA0006"  # Credential Access
  message: "Kill-chain SSH brute-force detectee depuis {src_ip}"
```

Architecture du moteur

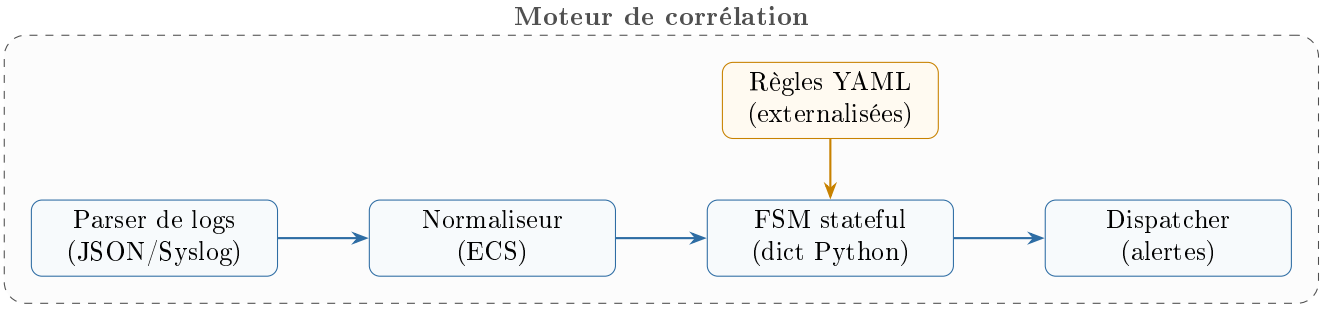

Figure 2. Modules internes du moteur de corrélation

Plan d'implémentation — 10 semaines

Table 5. Plan d'implémentation détaillé

Sem.	Phase	Tâches principales
0	Préparation	Dépôt Git + structure, schéma réseau, choix distributions, installation outils, décision VM/conteneur.
1–2	Topologie réseau IaC	Déploiement firewall Alpine, routage, segmentation. Docker Compose cibles + serveur collecte. Tests de connectivité.
3	Pipeline collecte	rsyslog + journald → collecte centralisée → normalisation JSON/ECS. **Isolation du dataset d'évaluation (30%).**
4	Moteur Python — base	Parser logs, machine à états (dict Python), fenêtres temporelles, chargement règles YAML. Tests unitaires.
5	Scénario 1 + métriques	Simulation brute-force SSH. Mesure TPR/FPR. Ajustement seuils.
6	Scénario 2 + Wazuh	Simulation scan→exfiltration. **Déploiement Wazuh.** Calibration FP. Collecte métriques comparatives.
7	Mini-SOC complet	Exposition alertes (fichier structuré/API). Mécanisme de réponse minimal (optionnel).
8	Benchmark Wazuh	Exécution des mêmes scénarios sur Wazuh. Mesure TPR, FPR, latence, RAM/CPU. Utilisation du dataset d'évaluation isolé.
9	Analyse & rapport	Comparaison chiffrée. Rédaction sections techniques. Visualisations.
10	Soutenance	Présentation + démonstration d'attaque/détection en direct (scénario 1 ou 2).

Règle d'or — Dataset d'évaluation

Le dataset d'évaluation (30 % des logs générés) est isolé en semaine 3, placé dans `datasets/eval/` et **jamais utilisé pendant le développement**. Il sert exclusivement à l'évaluation finale du moteur Python ET de Wazuh. Toute contamination de ce dataset invalide la comparaison et donc la validité scientifique du benchmark.

Métriques d'évaluation

Métriques de détection

Les métriques suivantes seront calculées pour chaque scénario, pour le moteur Python et pour Wazuh :

$$\text{TPR} = \frac{VP}{VP + FN} \quad \text{(sensibilité)} \tag{1}$$

$$\text{FPR} = \frac{FP}{FP + VN} \quad \text{(taux de faux positifs)} \tag{2}$$

$$\text{Précision} = \frac{VP}{VP + FP} \tag{3}$$

où VP = vrai positif, FN = faux négatif, FP = faux positif, VN = vrai négatif.

Métriques de performance

- **Latence de détection :** délai entre le dernier événement déclenchant et la génération de l'alerte (mesure en millisecondes, moyenne sur 20 répétitions par scénario).
- **RAM en charge :** consommation mémoire du processus Python ou du stack Wazuh pendant l'exécution des scénarios (via `/proc`).
- **CPU :** utilisation CPU moyenne sur la fenêtre de détection.

Objectifs de conception (valeurs cibles)

Rappel — Valeurs cibles, pas résultats

Le tableau ci-dessous présente des **objectifs de conception**. Les valeurs réelles mesurées les remplaceront dans le rapport final. Aucune de ces valeurs n'a été mesurée au moment de la rédaction de ce document.

Table 6. Objectifs de conception — à valider lors des tests

Métrique	Moteur Python	Wazuh	Source
TPR (%)	≥ 90	À mesurer	Objectif H1
FPR (%)	≤ 10	À mesurer	Objectif H1
Latence (ms)	$< 3\,000$	À mesurer	Objectif H2
RAM en charge	< 4 Go	À mesurer	Objectif H2

Impact et perspectives locales

Adéquation au contexte togolais

Ce projet répond à une contrainte réelle du tissu économique togolais : la nécessité de sécuriser des infrastructures modestes avec des moyens limités. Les choix techniques reflètent cette contrainte à chaque niveau :

- **Coût nul.** L'ensemble de la stack (Python, Docker, Alpine Linux, Wazuh) est open-source et ne nécessite aucune licence.
- **Empreinte mémoire maîtrisée.** Le moteur Python fonctionne sous 4 Go, compatible avec du matériel de seconde main couramment disponible au Togo.
- **Règles adaptables.** Les règles YAML peuvent être rédigées et modifiées par un développeur avec des compétences Python, sans formation SIEM spécialisée.

— **Scénarios représentatifs.** Les deux scénarios retenus (brute-force SSH, exfiltration) correspondent aux vecteurs d'attaque les plus fréquents dans les PME de la sous-région disposant de services exposés avec des credentials faibles.

Limites

— **Pas de scalabilité entreprise.** Le moteur Python en mémoire n'est pas conçu pour ingérer des milliers d'événements par seconde. Il cible explicitement les PME avec un trafic modéré.
— **Lab simulé vs production réelle.** Les métriques obtenues en laboratoire pourront différer du comportement en environnement de production réel (bruit de fond, anomalies inattendues).
— **Projet solo.** L'étendue du projet est limitée par le fait qu'un seul développeur en assure l'implémentation et les tests.

Perspectives

— Collaboration avec l'ANCy (Agence Nationale de la Cybersécurité) pour validation et éventuelle diffusion.
— Déploiement pilote au sein de l'EPL Lomé sur l'infrastructure réseau interne.
— Extension à d'autres scénarios de menaces mobiles (fraude Mobile Money, phishing WhatsApp) dans une version 2.
— Contribution à un référentiel de règles de détection contextualisées pour l'Afrique de l'Ouest.

Conclusion

Ce projet démontre la faisabilité d'une approche de détection d'intrusion stateful, low-cost et adaptée au contexte des PME togolaises. En construisant un moteur de corrélation Python avec des règles externalisées et en le benchmarquant rigoureusement contre Wazuh, il produit à la fois un livrable technique concret et une analyse comparative méthodologiquement solide.

Les compétences développées — architecture pipeline de données, programmation Python orientée sécurité, infrastructure as code, conduite de tests comparatifs — sont directement transférables à des rôles d'ingénieur SOC, d'analyste sécurité ou de consultant en cybersécurité, aussi bien au Togo que dans la sous-région ou à l'international.

Références

Agence Nationale de la Cybersécurité (ANCy). Rapport annuel sur l'état de la cybersécurité au togo, 2024. URL https://www.ancy.tg. Source à confirmer – consulter site officiel ANCy.

Elastic N.V. Elastic common schema (ecs) specification, 2024. URL https://www.elastic.co/guide/en/ecs/current/index.html.

Eric M. Hutchins, Michael J. Cloppert, and Rohan M. Amin. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. White paper, Lockheed Martin, 2011. URL

https://www.lockheedmartin.com/content/dam/lockheed-martin/rms/documents/cyber/LM-White-Paper-Intel-Driven-Defense.pdf.

International Telecommunication Union. Global cybersecurity index 2024, 2024. URL https://www.itu.int/en/ITU-D/Cybersecurity/Pages/global-cybersecurity-index.aspx. Classement Togo – section Afrique subsaharienne.

MITRE Corporation. ATT&CK enterprise matrix, 2024. URL https://attack.mitre.org/tactics/enterprise/.

République Togolaise – Ministère de l'Économie Numérique. Stratégie nationale de cybersécurité du togo 2024–2028, 2024. Document à sourcer officiellement.

Wazuh Inc. Wazuh documentation, 2024. URL https://documentation.wazuh.com.

Structure du dépôt Git

Listing 4. Structure du dépôt mini-soc-togo

```
mini-soc-togo/
|---- infra/
|    |---- docker-compose.yml       # Stack collecte + cibles
|    \---- firewall/                # Config Alpine iptables
|---- collector/
|    |---- rsyslog.conf             # Config collecte
|    \---- normalizer.py            # Pipeline JSON/ECS
|---- engine/
|    |---- core.py                  # Moteur FSM principal
|    |---- loader.py                # Chargement regles YAML
|    |---- rules/
|    |    |---- ssh_bruteforce.yaml # Scenario 1
|    |    \---- scan_exfil.yaml     # Scenario 2
|    \---- tests/
|         |---- test_ssh.py         # Tests unitaires S1
|         \---- test_scan.py        # Tests unitaires S2
|---- scenarios/
|    |---- run_scenario1.sh         # Script attaque S1
|    \---- run_scenario2.sh         # Script attaque S2
|---- datasets/
|    |---- dev/                     # 70% - developpement
|    \---- eval/                    # 30% - NE PAS MODIFIER
|---- wazuh/
|    \---- custom-rules.xml         # Regles Wazuh equivalentes
|---- results/
|    \---- metrics.csv              # Resultats benchmark
|---- report/
|    |---- main.tex                 # Ce document
|    \---- refs.bib                 # Bibliographie
\---- docs/
     |---- decisions.md             # Journal decisions archi
     \---- network-topology.md      # Topologie reseau
```

Règles de corrélation — Scénario 2

Listing 5. Règle YAML — Scénario 2 : scan → exfiltration

```yaml
# rules/scan_exfil.yaml
id: SCAN_EXFIL_001
name: "Reconnaissance reseau suivie d'une exfiltration"
description: >
  Detecte un scan reseau large depuis un hote interne
  suivi d'un transfert de donnees sortant vers une IP externe.
trigger:
  event_type: network_scan
  threshold: 20            # >= 20 hotes scanes
  window_seconds: 30       # en moins de 30 secondes
  group_by: src_ip
escalation:
  event_type: data_transfer_outbound
```

```
  same_field: src_ip
  min_bytes: 10000000      # >= 10 Mo
  within_seconds: 300      # dans les 5 minutes suivantes
alert:
  severity: CRITICAL
  mitre_tactic: "TA0010"  # Exfiltration
  message: "Kill-chain scan+exfiltration depuis {src_ip}"
```

Bibliographie — Sources à compléter

Sources à vérifier et compléter avant le rapport final

Les références ci-dessous sont des pointeurs à valider. Chaque entrée doit être vérifiée et remplacée par une source accessible et citable (rapport officiel, publication académique, site institutionnel).

Sources à investiguer :

- Rapports annuels de l'ANCy (Agence Nationale de la Cybersécurité du Togo)
- Rapports ITU sur l'index de cybersécurité mondial (GCI) — section Afrique
- Études CEDEAO sur la cybersécurité des PME en Afrique de l'Ouest
- Documentation officielle Wazuh : https://documentation.wazuh.com
- MITRE ATT&CK Enterprise : https://attack.mitre.org
- RFC 5424 (Syslog protocol), RFC 8259 (JSON)